Swirl

1. Filter ~2500 BLAST Hits
2. Subset the ref_seq file for Analysis.

Tutorial

When you see this '...' press enter after reading + when ready to continue.

bye() to exist + save progress.

skip()

Play()

main() returns to swirl main window

info() display options again

Numeric Vectors 4-1-26

Can be used in arithmetic expressions.

- Is a one-dimensional ordered collection of elements.

  either all #'s or all text

- Vectorization: perform a calculation on an entire vector all at once rather than looping through each element individually.

Atomic Vectors

Numeric: 1.5, 42

Integer: Whole # 5L

Character: Text strings

Logical: Boolean Values: TRUE, FALSE

Complex: 1+2i

Raw: ?

Lists

Recursive Vectors:

Create Vector

c() exp. x <- c(1, 2, 3)

Colon : exp. 1:10 A seq of #

Seq() exp. seq(from=1, to=10, by=2)

[For control over steps]

rep() exp. rep(1, times=5)

[to repeat values]

the vector of 2, was multiplied by 2. then added 100.

2 * 2 +100

Other Arithmetic Operators

x^2 = x squared

sqrt() = square root

abs() = absolute value

note → "<-" is to Assign a variable

Variables: a storage location used to hold data values that can be accessed, manipulated or updated throughout a program.

Technically are Also OBjects

my_sqrt contains A vector of 3.

type the variable and it will produce the value.

→ This will subtract 1 from each element in 2 then take the square root of each element

this = A vector of the same length as the original vector 2

R computes:

A) when there are two vectors of the same length, R does "+, -, *, etc element by element

B) if vectors are of different lengths, R recycles the shorter vector until it is the same length as the longer vector.

2 = 2 * 2+100] A vector of 3

BUT

"2+100" = 1 vector technically

So. z*2 +100 is .....

z* c(2, 2, 2) + c(100, 100, 100)

Vector Recycling

Add c(1, 2, 3, 4) and c(0, 10)

[Shorter vector is recycled.
R will provide A warning]

TRICKS

↑ = up arrow will cycle ①
through previous commands.

Change the value ②
or the mistake
and press enter

Type First
Two Letters
of variable
Name
hit TAB
(more than once?)

this provides
A List of
variables in your
workspace

Sequences of Numbers

lesson 3

: = An operator.

exp. 1:20] lists each # from 1 to 20: integers.

pi:10 = sequence of real #s.

output = A vector of real #s starting w/ pi

exp. 15:1] Backwards

To learn More About a function

Add function

?

For operators like ":"

Do this ?`:`

Backtick

seq() does the same as ":"

What if we want a vector of #s ranging from 0 to 10 increments by 0.5.

```
seq(0, 10, by=0.5)
```

```
seq(5, 10, length=30)
```

Want a seq of 30 #s between 5, 10

To Confirm The Length

```
length(my_seq)
```

Exercise

- Don't know the length of my_seq
- Want to generate a seq of integers from 1 to N

the length of the my_seq vector

- Basically want a new vector (1, 2, 3...) that is the same length as my_seq.

Do This: 1:length(my_seq)

OR

Do This: seq(along.with=my_seq)

CAN use

↓

seq_along()

Exp. seq_along(my_seq)

Creating Sequences of #s

rep() means replicate

Want to create a vector that contains 40 zeros.

```
rep(0, times=40)
```

Want our vector to contain 10 repititions of the vector (0,1,2)

```
rep(c(0,1,2), times=10)
```

Want our vector to contain 10 zeros, then 10 ones, then 10 twos.

```
rep(c(0,1,2), each=10)
```

Vectors

most common data structure

↓ Atomic → Only one data type

↓ Lists → Multiple data Types

Logical Vectors: TRUE, FALSE, NA

Create a variable:

num_vect <- c(0.5, 55, -10, 6)

Create a variable where tF gets the result of num_vect < 1

tF <- num_vect < 1

tF view is = TRUE FALSE TRUE FALSE

num_vect < 1
- is a condition.
- tF tells us whether each corresponding element of our numeric vector num_vect satisfies this condition.

So

TRUE	FALSE	TRUE	FALSE
0.5	55	-10	6

Exercise

num_vect >= 6 → greater or equal to 6

FALSE	TRUE	FALSE	TRUE
0.5	55	-10	6

Logical Operators

'<' and '>='

others: '>', '<=', '==' → exact equality

'!=' → inequality

If we have

TWO LOGICAL expressions

Ask) is one True?] A | B (logical 'or' 'union')

OR

if both are true] A & B (logical 'and' a.k.a. 'intersection')

! A] the negation of A and is TRUE when A is FALSE and vice versa

Examples:

(3 > 5) & (4 == 4) A

answer is False

Atleast one of the two is true

(TRUE == TRUE) | (TRUE == FALSE) B

Answer is True

C

(111 >= 111) | ! (TRUE)) & ((4+1) == 5)

Answer is true

Character Vectors: " "

my_char <- c("My", "name", "is")

my_char

is a character vector of length = 3

Exercise:
We want to join the elements of my_char together into one continous character string: A character vector of length 1

Use paste()

TRY: paste(my_char, collapse = " ")

Output: "My name is" not "My" "name" "is"

Now collapse the quotations.

TRY This

Paste(my_name, collapse = " ")

Output = "My name is Norbaya"

Add or Concatenate

your name to the end of my_char
use c() like this.

c(my_char, "your_name_here")
my_name
Output: "My" "name" "is" "Norbaya"

Join Names w/ paste()

Join Two Character Vectors

that are each length of 1

meaning: join two words

space

Paste("Hello", "world!", sep = " ")

Output

"Hello world!"

tells R to seperate the joined elements w/ a single space.

CAN Join Two Vectors

each of length 3

Paste(1:3, c("x", "y", "z"), sep = " ")

Due to different vector lengths,

Recyle!

TRY] Paste(LETTERS, 1:4, sep="-")

in this case: 1, 2, 3, 4 are not numbers
but characters "1" "2" "3" "4"

Missing Values

NA = not available or missing

x.3 output: x <- c(44, NA, 5, NA)

output: 132, NA, 15, NA

Vector w/ 1000 draws from a std normal distribution

① y <- rnorm(1000)

② z <- rep(NA, 1000)

③ Select 100 elements at random from the 2000 values (combining y and z)

my_data <- sample(c(y, z), 100)

④ Call is.na()

to know if each element of a vector is NA.

my_na <- is.na(my_data)

my_na

TRUE means = na

"==" means, a method of testing for equality between two objects.

my_data == NA

TRUE = 1 FALSE = 0

The sum of both = total # of TRUEs

sum(my_na)

my_data

NaN means, not a number

$0/0 = NaN$

$Inf - Inf =$

Subsetting Vectors

How to extract elements from a vector based on some conditions

if we're only interested in the 1st 20 elements of a vector or only the elements that are not NA

> if you want to select some particular elements from a vector by placing an index vector in [ ] after following the vector name.

X [1:10]

is.na(x) yields a vector of logical values the same length of x

X [is.na(x)] | A vector of all NAs

Remember: ! gives the negation of logical expression.

!is.na(x) = is not na

y <- x[!is.na(x)] = all values not na stored in y

y > 0 = vector w/ logical values the same length as y w/ TRUEs corresponding to the values of y

y[y>0] = all y values greater than 0

So, x[!is.na(x) & x>0]

↳ values not missing and greater than zero

1st element of a vector = element 1

To look for 3rd, 5th, 7th element in a numeric vector

x[c(3,5,7)]

x[3000] = NONE!

Ask for what is written w/in the bounds of the vector

What if we want all elements

Except 2nd + 10th

① $x[c(2,10)]$ = ONLY 2nd + 10th elements of x

② $x[c(-2,-10)]$ ALL except 2nd + 10 elements

Multiple Neg #

Short Hand

$x[-c(2,10)]$

Named Elements

vect <- c(foo=11, bar=2, norf=NA)

foo	bar	norf
11	2	NA

OR

names(vect) = "foo" "bar" "norf"

OR

vect2 <- c(11, 2, NA)

Then add Names Attribute to

names(vect2) <- c("foo", "bar", "norf")

Are they identical?

identical(vect, vec2)

vect["bar"] = 2

Matrices + Data Frames

"rectangular" data type.
Stores tabular data w/ rows + columns

matrices = single class of data
data frames = different classes of data

Exercise

```
my_vector <- 1:20
```

```
dim (my_vector)
```

dim() = dimension of an object

Output = NULL

```
length (my_vector)
```

Output = 20

What if we give it a dim() attribute

```
dim (my_vector) <- c(4,5)
```

dim()
Also allows us to
get OR set

a dim attribute for
an R object

```
dim(my_vector)
```

Attributes(my_vector)

is now a matrix · 1st # = Row

my_vector 2nd # = Column

Class(my_vector) to confirm the class type

Output: class(my_vector)

matrix() to create more direct matrix

?Matrix For help

Create a matrix w/ #s 1-20 and w/ dimensions of (4 rows, 5 clums)

my_matrix2 <- matrix(1:20, nrow=4, ncol=5)

identical(my_matrix, mymatrix2)

LABEL the ROWS

So we know which # belong to each Patient in the experiment

" " mean something is a string!

Patients <- ("Bill", "Gina", "Kelly", "Sean")

cbind() combines columns

cbind(patients, matrix)

Data Issue -> R made each number into characters "implicit coercion"

Try:

```
my_data <- data.frame(patients, my_matrix)

my_data
```

data.frame() takes any # of arguments + returns a single object of class data.frame composed of original objects

Confirm with

```
class(my_data)
```

Assign Names to Columns of our data Frames

① Create a vector w/ all names

```
cnames <- c("patient", "age", "weight", "bp", "cardig",
            "test")
```

② Use colnames() to set the colnames attribute for our data Frame.

```
colnames(my_data) <- cnames

my_data
```

Logic

Functions

Sys.Date()

mean() Average

mean(c(2,4,5))

boring_function <- function(x){
x
}

① everything that exists is an object.

② everything that happens is a function call

my_mean <- function(my_vector){
sum(my_vector) / length(my_vector)
}

```
increment 2-Function (number, by=1{
    number +by
}
```

default value of 1

Set default values for a functions argument

```
remainder 2-Function (num, divisor=2){
    num % divisor
}
```

Ex: remainder (divisor=11, num=5)

args (remainder) to see the arguments

Looking at Data

ls() to list variables in workspace

class(plants)

dim(plants)

nrow(plants)

ncol(plants)

object.size(plants)

names(plants)

head(plants) to preview top of data set

head(plants, 10) 1st 6 rows

tail(plants, 15) last row

Summary(Plants)

table(plants $ Active_Growth_Period)

↳ categorical

str(Plants) to understand the structure

↳ structure of data

Save script = ctrl-S